

What is Advance Java?

The dictionary meaning of **advance** is a forward movement or a development or improvement and the meaning of improve means thing that makes something better. We have to improve our basic knowledge to master in that particular field.

- [Java](#) is divided into two parts i.e. **Core Java (J2SE)** and **Advanced Java (JEE)**.
- The core Java part covers the fundamentals (data types, functions, operators, loops, thread, exception handling, etc.) of the Java programming language. It is used to develop general purpose applications.
- Whereas **Advanced Java** covers the standard concepts such as database connectivity, networking, Servlet, web-services, etc.

Advance Java

- It is a part of Java programming language.
- It is an advanced technology or advance version of Java specially designed to develop web-based, network-centric or enterprise applications.
- It includes the concepts like [Servlet](#), [JSP](#), JDBC, [RMI](#), [Socket programming](#), etc. It is a specialization in specific domain.
- Most of the applications developed using advance Java uses tow-tier architecture i.e. Client and Server.
- All the applications that runs on Server can be considered as advance Java applications.

Why advance Java?

- It simplifies the complexity of a building n-tier application.
- Standardizes and API between components and application sever container.
- JEE application Server and Containers provides the framework services.

Benefits of Advance Java

The four major benefits of advance Java that are, network centric, process simplification, and futuristic imaging standard.

- JEE (advance Java) provides libraries to understand the concept of **Client-Server architecture** for web- based applications.

- We can also work with web and application servers such as **Apache Tomcat** and **Glassfish**. Using these servers, we can understand the working of HTTP protocol. It cannot be done in core Java.
- It is also important to understand the advanced Java if you are dealing with trading technologies like **Hadoop**, **cloud-native** and **data science**.
- It provides a set of services, **API** and **protocols**, that provides the functionality which is necessary for developing **multi-tiered** application, web-based application.
- There is a number of advanced Java frameworks like, **Spring**, **Hibernate**, **Struts**, that enables us to develop secure **transaction-based** web applications such as banking application, inventory management application.

Difference between Core Java and Advance Java

Criteria	Core Java	Advance Java
Used for	It is used to develop general purpose application.	It is used to develop web-based applications.
Purpose	It does not deal with database, socket programming, etc.	It deals with socket programming, DOM, and networking applications.
Architecture	It is a single tier architecture.	It is a multi-tier architecture.
Edition	It is a Java Standard Edition.	It is a Java Enterprise Edition.
Package	It provides java.lang.* package.	It provides java.servlet.* package.

Java JDBC Tutorial

- JDBC stands for Java Database Connectivity.
- JDBC is a Java API to connect and execute the query with the database.
- JDBC is a part of JavaSE (Java Standard Edition).
- JDBC API uses JDBC drivers to connect with the database.
- JDBC API use to access tabular data stored in any relational database.
- By the help of JDBC API, we can save, update, delete and fetch data from the database. It is like Open Database Connectivity (ODBC) provided by Microsoft.
- The current version of JDBC is 4.3. It is the stable release since 21st September, 2017. It is based on the X/Open SQL Call Level Interface.
- The **java.sql** package contains classes and interfaces for JDBC API.

A list of popular *interfaces* of JDBC API are given below:

- Driver interface
- Connection interface
- Statement interface
- PreparedStatement interface
- CallableStatement interface
- ResultSet interface
- ResultSetMetaData interface
- DatabaseMetaData interface
- RowSet interface

A list of popular *classes* of JDBC API are given below:

- DriverManager class
- Blob class
- Clob class
- Types class

Why Should We Use JDBC

Before JDBC, ODBC API was the database API to connect and execute the query with the database. But, ODBC API uses ODBC driver which is written in C language (i.e. platform dependent and unsecured). That is why Java has defined its own API (JDBC API) that uses JDBC drivers (written in Java language).

We can use JDBC API to handle database using Java program and can perform the following activities:

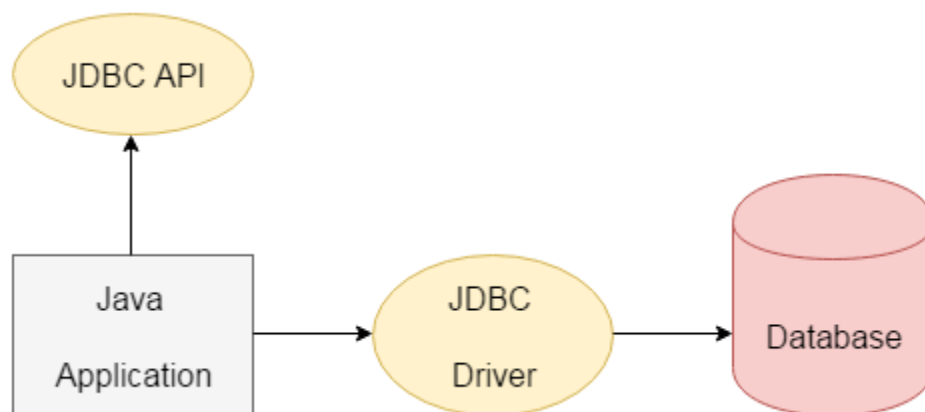
1. Connect to the database
2. Execute queries and update statements to the database
3. Retrieve the result received from the database.

What is API

API (Application programming interface) is a document that contains a description of all the features of a product or software. It represents classes and interfaces that software programs can follow to communicate with each other. An API can be created for applications, libraries, operating systems, etc.

JDBC Driver

JDBC Driver is a software component that enables java application to interact with the database.



There are 4 types of JDBC drivers:

1. JDBC-ODBC bridge driver
2. Native-API driver (partially java driver)
3. Network Protocol driver (fully java driver)
4. Thin driver (fully java driver)

1) JDBC-ODBC bridge driver

- The JDBC-ODBC bridge driver uses ODBC driver to connect to the database.
- The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls.

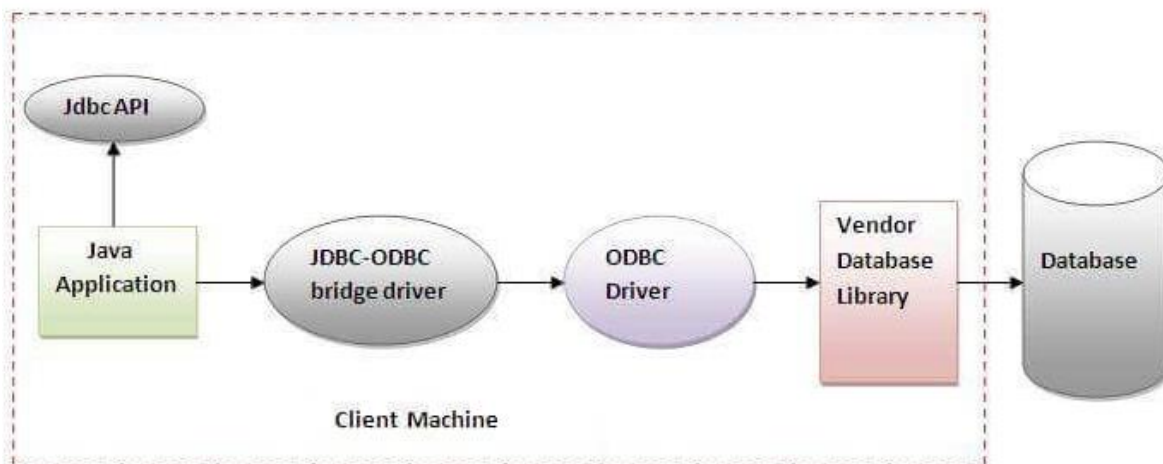


Figure- JDBC-ODBC Bridge Driver

In Java 8, the JDBC-ODBC Bridge has been removed.

Oracle does not support the JDBC-ODBC Bridge from Java 8. Oracle recommends that you use JDBC drivers provided by the vendor of your database instead of the JDBC-ODBC Bridge.

Advantages:

- easy to use.
- can be easily connected to any database.

Disadvantages:

- Performance degraded because JDBC method call is converted into the ODBC function calls.
- The ODBC driver needs to be installed on the client machine.

2) Native-API driver

- The Native API driver uses the client-side libraries of the database.

- The driver converts JDBC method calls into native calls of the database API. It is not written entirely in java.

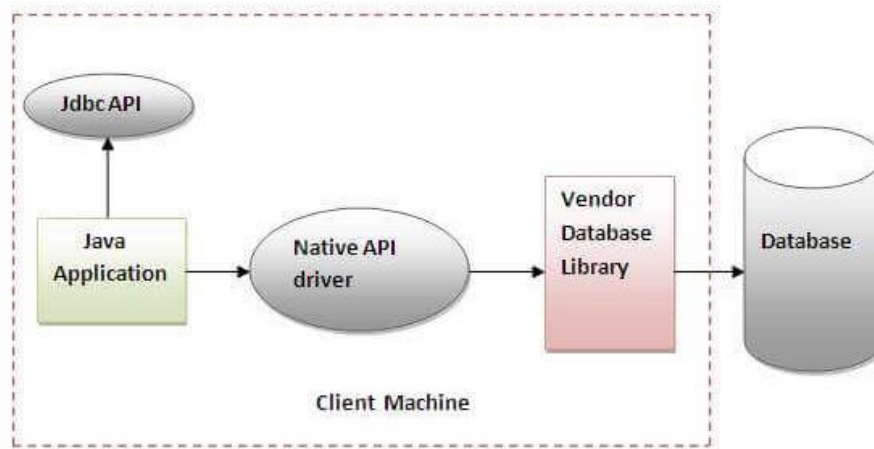


Figure- Native API Driver

Advantage:

- performance upgraded than JDBC-ODBC bridge driver.

Disadvantage:

- The Native driver needs to be installed on the each client machine.
- The Vendor client library needs to be installed on client machine.

3) Network Protocol driver

The Network Protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol. It is fully written in java.

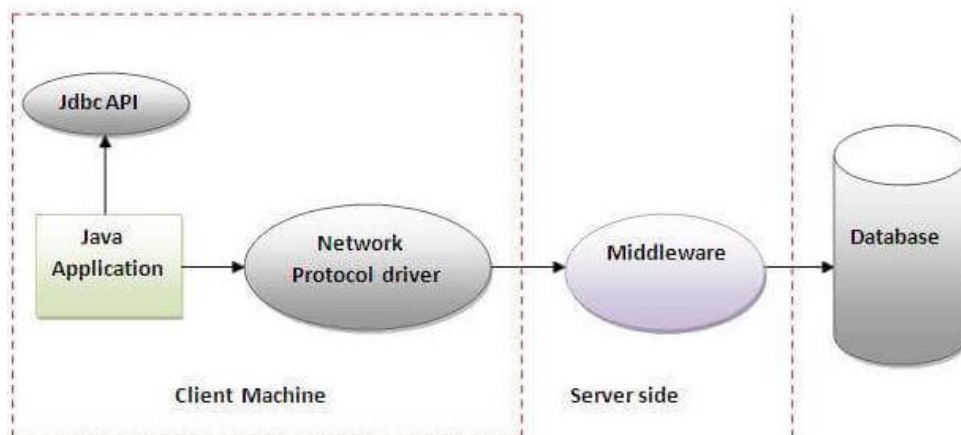


Figure- Network Protocol Driver

Advantage:

- No client side library is required because of application server that can perform many tasks like auditing, load balancing, logging etc.

Disadvantages:

- Network support is required on client machine.
- Requires database-specific coding to be done in the middle tier.
- Maintenance of Network Protocol driver becomes costly because it requires database-specific coding to be done in the middle tier.

4) Thin driver

The thin driver converts JDBC calls directly into the vendor-specific database protocol. That is why it is known as thin driver. It is fully written in Java language.

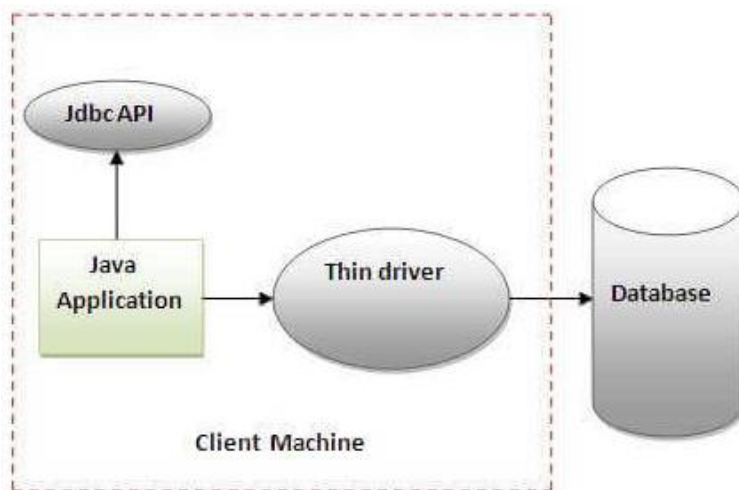


Figure- Thin Driver

Advantage:

- Better performance than all other drivers.
- No software is required at client side or server side.

Disadvantage:

- Drivers depend on the Database.

Java Database Connectivity with 5 Steps

There are 5 steps to connect any java application with the database using JDBC. These steps are as follows:

- Register the Driver class
- Create connection
- Create statement
- Execute queries
- Close connection

Java Database Connectivity



1) Register the driver class

The **forName()** method of Class class is used to register the driver class. This method is used to dynamically load the driver class.

Syntax of forName() method

1. **public static void** forName(String className)**throws** ClassNotFoundException

Note: Since JDBC 4.0, explicitly registering the driver is optional. We just need to put vender's Jar in the classpath, and then JDBC driver manager can detect and load the driver automatically.

Example to register the OracleDriver class

Here, Java program is loading oracle driver to establish database connection.

1. `Class.forName("oracle.jdbc.driver.OracleDriver");`

2) Create the connection object

The **getConnection()** method of DriverManager class is used to establish connection with the database.

Syntax of getConnection() method

- 1) **public static** Connection getConnection(String url)**throws** SQLException
- 2) **public static** Connection getConnection(String url,String name,String password)
3. **throws** SQLException

Example to establish connection with the Oracle database

1. Connection con=DriverManager.getConnection(
2. "jdbc:oracle:thin:@localhost:1521:xe","system","password");

3) Create the Statement object

The createStatement() method of Connection interface is used to create statement. The object of statement is responsible to execute queries with the database.

Syntax of createStatement() method

1. **public** Statement createStatement()**throws** SQLException

Example to create the statement object

1. Statement stmt=con.createStatement();

4) Execute the query

The executeQuery() method of Statement interface is used to execute queries to the database. This method returns the object of ResultSet that can be used to get all the records of a table.

Syntax of executeQuery() method

1. **public** ResultSet executeQuery(String sql)**throws** SQLException

Example to execute query

1. ResultSet rs=stmt.executeQuery("select * from emp");
- 2.
3. **while**(rs.next()){
4. System.out.println(rs.getInt(1)+" "+rs.getString(2));
5. }

5) Close the connection object

By closing connection object statement and ResultSet will be closed automatically. The close() method of Connection interface is used to close the connection.

Syntax of close() method

1. **public void** close()**throws** SQLException

Example to close connection

1. con.close();

Note: Since Java 7, JDBC has ability to use try-with-resources statement to automatically close resources of type Connection, ResultSet, and Statement.

It avoids explicit connection closing step.

Java Database Connectivity with MySQL

To connect Java application with the MySQL database, we need to follow 5 following steps.

In this example we are using MySql as the database. So we need to know following informations for the mysql database:

1. **Driver class:** The driver class for the mysql database is **com.mysql.jdbc.Driver**.
2. **Connection URL:** The connection URL for the mysql database is **jdbc:mysql://localhost:3306/sonoo** where jdbc is the API, mysql is the database, localhost is the server name on which mysql is running, we may also use IP address, 3306 is the port number and sonoo is the database name. We may use any database, in such case, we need to replace the sonoo with our database name.
3. **Username:** The default username for the mysql database is **root**.
4. **Password:** It is the password given by the user at the time of installing the mysql database. In this example, we are going to use root as the password.

Let's first create a table in the mysql database, but before creating table, we need to create database first.

1. create database sonoo;
2. use sonoo;
3. create table emp(id **int**(10),name varchar(40),age **int**(3));

Example to Connect Java Application with mysql database

In this example, sonoo is the database name, root is the username and password both.

```
1. import java.sql.*;
2. class MysqlCon{
3.     public static void main(String args[]){
4.         try{
5.             Class.forName("com.mysql.jdbc.Driver");
6.             Connection con=DriverManager.getConnection(
7.                 "jdbc:mysql://localhost:3306/sonoo","root","root");
8.             //here sonoo is database name, root is username and password
9.             Statement stmt=con.createStatement();
10.            ResultSet rs=stmt.executeQuery("select * from emp");
11.            while(rs.next())
12.                System.out.println(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getString(3));
13.            con.close();
14.        }catch(Exception e){ System.out.println(e);}
15.    }
16. }
```

The above example will fetch all the records of emp table.

To connect java application with the mysql database, **mysqlconnector.jar** file is required to be loaded.

[download the jar file mysql-connector.jar](#)

Two ways to load the jar file:

1. Paste the mysqlconnector.jar file in jre/lib/ext folder
2. Set classpath

1) Paste the mysqlconnector.jar file in JRE/lib/ext folder:

Download the mysqlconnector.jar file. Go to jre/lib/ext folder and paste the jar file here.

2) Set classpath:

There are two ways to set the classpath:

- temporary
- permanent

How to set the temporary classpath

open command prompt and write:

1. C:>set classpath=c:\folder\mysql-connector-java-5.0.8-bin.jar;.

How to set the permanent classpath

Go to environment variable then click on new tab. In variable name write **classpath** and in variable value paste the path to the mysqlconnector.jar file by appending mysqlconnector.jar;. as C:\folder\mysql-connector-java-5.0.8-bin.jar;.

Components of JDBC

JDBC has **four** main components that are used to connect with a database as follows:

1. JDBC API
2. JDBC Driver Manager
3. JDBC Test Suite
4. JDBC-ODBC Bridge Drivers

JDBC API

JDBC API is a collection of various classes, methods, and interfaces for easy communication with a database. JDBC API provides two different packages to connect with different databases.

```
java.sql.*;  
javax.sql.*;
```

JDBC Driver Manager

JDBC DriverManager is a **class in JDBC API** that loads a **database-specific driver** in Java application and establishes a connection with a database. DriverManager makes a call to a specific database to process the user request.

JDBC Test Suite

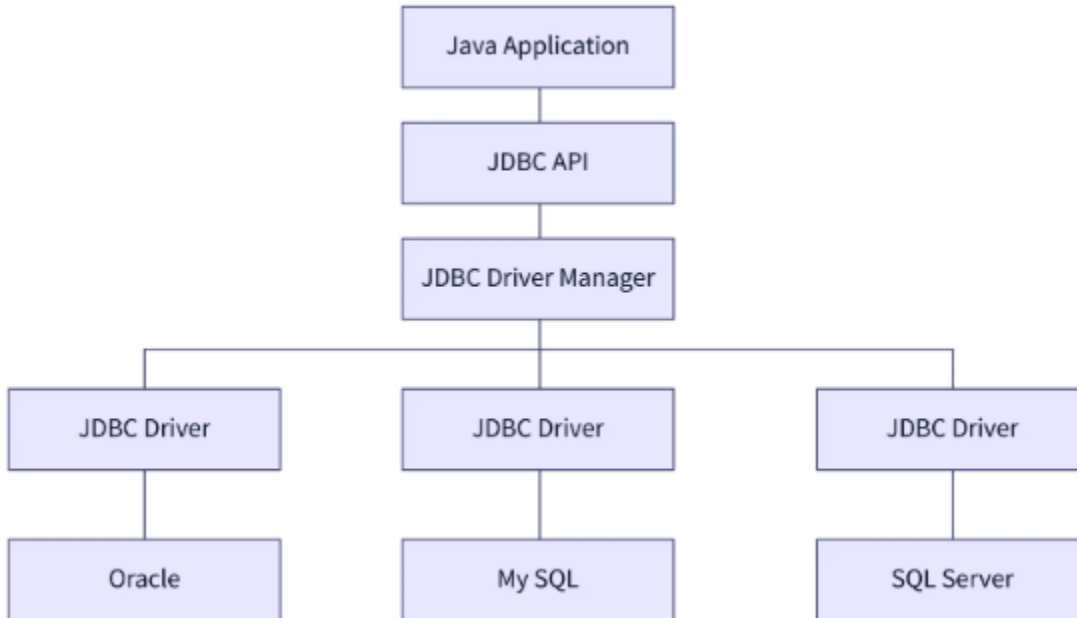
JDBC Test Suite is used to **test operations** being performed on databases using JDBC drivers. JDBC Test Suite tests operations such as insertion, updating, and deletion.

JDBC-ODBC Bridge Drivers

As the name suggests JDBC-ODBC Bridge Drivers are used to translate JDBC methods to **ODBC function calls**. JDBC-ODBC Bridge Drivers are used to connect database drivers to the database. Even after using JDBC for Java Enterprise Applications, we need an ODBC connection for connecting with databases.

JDBC-ODBC Bridge Drivers are used to bridge the same gap between JDBC and ODBC drivers. The bridge translates the object-oriented JDBC method call to the procedural ODBC function call. It makes use of the ***sun.jdbc.odbc*** package. This package includes a native library to access ODBC characteristics.

Architecture of JDBC



As we can see in the above image the major components of JDBC architecture are as follows:

1. Application
2. The JDBC API
3. DriverManager
4. JDBC Drivers
5. Data Sources

Application

Applications in JDBC architecture are java applications like applets or servlet that communicates with databases.

JDBC API

The JDBC API is an Application Programming Interface used to **create Databases**. JDBC API uses classes and interfaces to connect with databases. Some of the important classes and interfaces defined in JDBC architecture in java are the DriverManager class, Connection Interface, etc.

DriverManager

DriverManager class in the JDBC architecture is used to **establish a connection** between Java applications and databases. Using the `getConnection` method of this class a connection is established between the Java application and data sources.

JDBC Drivers

JDBC drivers are used to connecting with data sources. All databases like Oracle, MSSQL, MYSQL, etc. have their drivers, to connect with these databases we need to load their specific drivers. **Class** is a java class used to load drivers. **Class.forName()** method is used to load drivers in JDBC architecture.

Data Sources

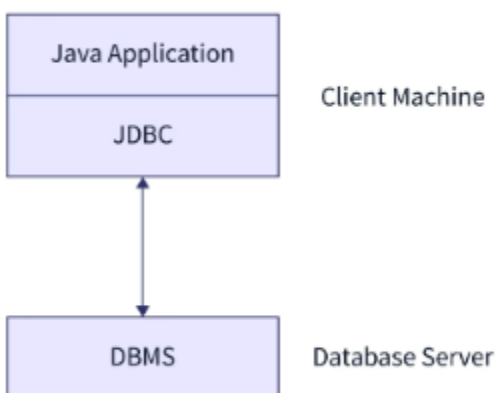
Data Sources in the JDBC architecture are the databases that we can connect using this API. These are the sources where data is stored and used by Java applications. JDBC API helps to connect various databases like Oracle, MYSQL, MSSQL, PostgreSQL, etc.

Types of JDBC Architecture

The JDBC Architecture can be of two types based on the processing models it uses. These models are

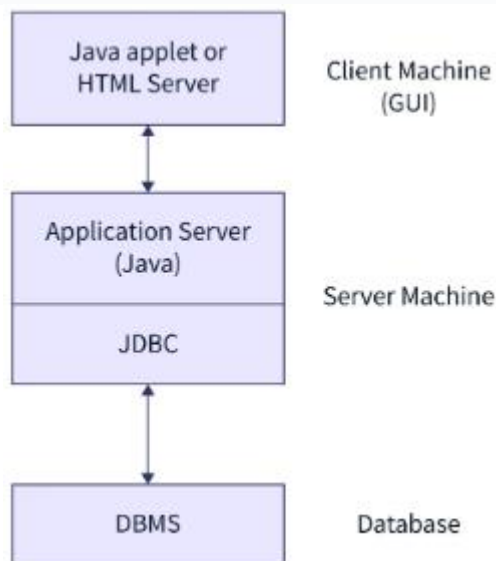
1. 2-tier model
2. 3-tier model

2 Tier Model



- 2-tier JDBC architecture model is a **basic model**.
- In this model, a java application communicates directly to the data sources. JDBC driver is used to establish a connection between the application and the data source.
- When an application needs to interact with a database, a query is directly executed on the data source and the output of the queries is sent back to the user in form of results.
- In this model, the data source can be located on a different machine connected to the same network the user is connected to.
- This model is also known as a **client/server** configuration. Here user's machine acts as a client and the machine on which the database is located acts as a server.

3 Tier Model



- 3-tier model is a **complex and more secure model** of JDBC architecture in java.
- In this model the user queries are sent to the middle tier and then they are executed on the data source.
- Here, the java application is considered as one tier connected to the data source(3rd tier) using middle-tier services.
- In this model user queries are sent to the data source using middle-tier services, from where the commands are again sent to databases for execution.
- The results obtained on the database are again sent to the middle-tier and then to the user/application.

DriverManager class

- The DriverManager class is the component of JDBC API and also a member of the *java.sql* package.
- The DriverManager class acts as an interface between users and drivers.
- It keeps track of the drivers that are available and handles establishing a connection between a database and the appropriate driver.
- It contains all the appropriate methods to register and deregister the database driver class and to create a connection between a Java application and the database.
- The DriverManager class maintains a list of Driver classes that have registered themselves by calling the method `DriverManager.registerDriver()`.
- Note that before interacting with a Database, it is a mandatory process to register the driver; otherwise, an exception is thrown.

Methods of the DriverManager Class

Method	Description
1) public static synchronized void registerDriver(Driver driver):	is used to register the given driver with DriverManager. No action is performed by the method when the given driver is already registered.
2) public static synchronized void deregisterDriver(Driver driver):	is used to deregister the given driver (drop the driver from the list) with DriverManager. If the given driver has been removed from the list, then no action is performed by the method.
3) public static Connection getConnection(String url) throws SQLException:	is used to establish the connection with the specified url. The SQLException is thrown when the corresponding Driver class of the given database is not registered with the DriverManager.
4) public static Connection getConnection(String url,String username,String password) throws SQLException:	is used to establish the connection with the specified url, username, and password. The SQLException is thrown when

userName,String password) throws SQLException:	the corresponding Driver class of the given database is not registered with the DriverManager.
5) public static Driver getDriver(String url)	Those drivers that understand the mentioned URL (present in the parameter of the method) are returned by this method provided those drivers are mentioned in the list of registered drivers.
6) public static int getLoginTimeout()	The duration of time a driver is allowed to wait in order to establish a connection with the database is returned by this method.
7) public static void setLoginTimeout(int sec)	The method provides the time in seconds. sec mentioned in the parameter is the maximum time that a driver is allowed to wait in order to establish a connection with the database. If 0 is passed in the parameter of this method, the driver will have to wait infinitely while trying to establish the connection with the database.
8) public static Connection getConnection(String URL, Properties prop) throws SQLException	A connection object is returned by this method after creating a connection to the database present at the mentioned URL, which is the first parameter of this method. The second parameter, which is "prop", fetches the authentication details of the database (username and password.). Similar to the other variation of the getConnection() method, this method also throws the SQLException, when the corresponding Driver class of the given database is not registered with the DriverManager.

Connection interface

- A Connection is a session between a Java application and a database.
- It helps to establish a connection with the database.
- The Connection interface is a factory of Statement, PreparedStatement, and DatabaseMetaData, i.e., an object of Connection can be used to get the object of Statement and DatabaseMetaData.
- The Connection interface provide many methods for transaction management like commit(), rollback(), setAutoCommit(), setTransactionIsolation(), etc.

By default, connection commits the changes after executing queries.

Commonly used methods of Connection interface:

- 1) **public Statement createStatement():** creates a statement object that can be used to execute SQL queries.
- 2) **public Statement createStatement(int resultSetType,int resultSetConcurrency):** Creates a Statement object that will generate ResultSet objects with the given type and concurrency.
- 3) **public void setAutoCommit(boolean status):** is used to set the commit status. By default, it is true.
- 4) **public void commit():** saves the changes made since the previous commit/rollback is permanent.
- 5) **public void rollback():** Drops all changes made since the previous commit/rollback.
- 6) **public void close():** closes the connection and Releases a JDBC resources immediately.

Connection Interface Fields

There are some common Connection interface constant fields that are present in the Connect interface. These fields specify the isolation level of a transaction.

TRANSACTION_NONE: No transaction is supported, and it is indicated by this constant.

TRANSACTION_READ_COMMITTED: It is a constant which shows that the dirty reads are not allowed. However, phantom reads and non-repeatable reads can occur.

TRANSACTION_READ_UNCOMMITTED: It is a constant which shows that dirty reads, non-repeatable reads, and phantom reads can occur.

TRANSACTION_REPEATABLE_READ: It is a constant which shows that the non-repeatable reads and dirty reads are not allowed. However, phantom reads and can occur.

TRANSACTION_SERIALIZABLE: It is a constant which shows that the non-repeatable reads, dirty reads as well as the phantom reads are not allowed.

Statement interface

The **Statement interface** provides methods to execute queries with the database. The statement interface is a factory of ResultSet i.e. it provides factory method to get the object of ResultSet.

Commonly used methods of Statement interface:

The important methods of Statement interface are as follows:

- 1) public ResultSet executeQuery(String sql):** is used to execute SELECT query. It returns the object of ResultSet.
- 2) public int executeUpdate(String sql):** is used to execute specified query, it may be create, drop, insert, update, delete etc.
- 3) public boolean execute(String sql):** is used to execute queries that may return multiple results.
- 4) public int[] executeBatch():** is used to execute batch of commands.

Example of Statement interface

Let's see the simple example of Statement interface to insert, update and delete the record.

1. **import** java.sql.*;
2. **class** FetchRecord{
3. **public static void** main(String args[])**throws** Exception{
4. Class.forName("oracle.jdbc.driver.OracleDriver");
5. Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system",
"oracle");
6. Statement stmt=con.createStatement();
- 7.
8. `//stmt.executeUpdate("insert into emp765 values(33,'Irfan',50000)");`
9. `//int result=stmt.executeUpdate("update emp765 set name='Vimal',salary=10000 where id=33");`
10. **int** result=stmt.executeUpdate("delete from emp765 where id=33");
11. System.out.println(result+" records affected");
12. con.close();

13. }}

ResultSet interface

The object of ResultSet maintains a cursor pointing to a row of a table. Initially, cursor points to before the first row.

By default, ResultSet object can be moved forward only and it is not updatable.

But we can make this object to move forward and backward direction by passing either TYPE_SCROLL_INSENSITIVE or TYPE_SCROLL_SENSITIVE in createStatement(int,int) method as well as we can make this object as updatable by:

1. Statement stmt = con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
2. ResultSet.CONCUR_UPDATABLE);

Commonly used methods of ResultSet interface

1) public boolean next():	is used to move the cursor to the one row next from the current position.
2) public boolean previous():	is used to move the cursor to the one row previous from the current position.
3) public boolean first():	is used to move the cursor to the first row in result set object.
4) public boolean last():	is used to move the cursor to the last row in result set object.
5) public boolean absolute(int row):	is used to move the cursor to the specified row number in the ResultSet object.
6) public boolean relative(int row):	is used to move the cursor to the relative row number in the ResultSet object, it may be positive or negative.
7) public int getInt(int columnIndex):	is used to return the data of specified column index of the current row as int.
8) public int getInt(String columnName):	is used to return the data of specified column name of the current row as int.

9) public String getString(int columnIndex):	is used to return the data of specified column index of the current row as String.
10) public String getString(String columnName):	is used to return the data of specified column name of the current row as String.

Example of Scrollable ResultSet

Let's see the simple example of ResultSet interface to retrieve the data of 3rd row.

```

1. import java.sql.*;
2. class FetchRecord{
3.     public static void main(String args[])throws Exception{
4.
5.         Class.forName("oracle.jdbc.driver.OracleDriver");
6.         Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system",
            "oracle");
7.         Statement stmt=con.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,ResultSet.CONC
            UR_UPDATABLE);
8.         ResultSet rs=stmt.executeQuery("select * from emp765");
9.
10.        //getting the record of 3rd row
11.        rs.absolute(3);
12.        System.out.println(rs.getString(1)+" "+rs.getString(2)+" "+rs.getString(3));
13.
14.        con.close();
15.    }}

```

PreparedStatement interface

The PreparedStatement interface is a subinterface of Statement. It is used to execute parameterized query.

Let's see the example of parameterized query:

```

1. String sql="insert into emp values(?,?,?)";

```

As you can see, we are passing parameter (?) for the values. Its value will be set by calling the setter methods of PreparedStatement.

Why use PreparedStatement?

Improves performance: The performance of the application will be faster if you use PreparedStatement interface because query is compiled only once.

How to get the instance of PreparedStatement?

The prepareStatement() method of Connection interface is used to return the object of PreparedStatement. Syntax:

1. **public** PreparedStatement prepareStatement(String query)**throws** SQLException{ }

Methods of PreparedStatement interface

The important methods of PreparedStatement interface are given below:

Method	Description
public void setInt(int paramIndex, int value)	sets the integer value to the given parameter index.
public void setString(int paramIndex, String value)	sets the String value to the given parameter index.
public void setFloat(int paramIndex, float value)	sets the float value to the given parameter index.
public void setDouble(int paramIndex, double value)	sets the double value to the given parameter index.
public int executeUpdate()	executes the query. It is used for create, drop, insert, update, delete etc.
public ResultSet executeQuery()	executes the select query. It returns an instance of ResultSet.

Example of PreparedStatement interface that inserts the record

First of all create table as given below:

1. create table emp(id number(10),name varchar2(50));

Now insert records in this table by the code given below:

```
1. import java.sql.*;
2. class InsertPrepared{
3.     public static void main(String args[]){
4.         try{
5.             Class.forName("oracle.jdbc.driver.OracleDriver");
6.
7.             Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system",
               "oracle");
8.
9.             PreparedStatement stmt=con.prepareStatement("insert into Emp values(?,?)");
10.            stmt.setInt(1,101);//1 specifies the first parameter in the query
11.            stmt.setString(2,"Ratan");
12.
13.            int i=stmt.executeUpdate();
14.            System.out.println(i+" records inserted");
15.
16.            con.close();
17.
18.        }catch(Exception e){ System.out.println(e);}
19.
20.    }
21. }
```

Example of PreparedStatement interface that updates the record

```
1. PreparedStatement stmt=con.prepareStatement("update emp set name=? where id=?");
2. stmt.setString(1,"Sonoo");//1 specifies the first parameter in the query i.e. name
3. stmt.setInt(2,101);
4.
5. int i=stmt.executeUpdate();
```


6. `System.out.println(i+" records updated");`

Example of PreparedStatement interface that deletes the record

1. `PreparedStatement stmt=con.prepareStatement("delete from emp where id=?");`
 2. `stmt.setInt(1,101);`
 - 3.
 4. `int i=stmt.executeUpdate();`
 5. `System.out.println(i+" records deleted");`
-

Example of PreparedStatement interface that retrieve the records of a table

1. `PreparedStatement stmt=con.prepareStatement("select * from emp");`
 2. `ResultSet rs=stmt.executeQuery();`
 3. `while(rs.next()){`
 4. `System.out.println(rs.getInt(1)+" "+rs.getString(2));`
 5. `}`
-

Example of PreparedStatement to insert records until user press n

1. `import java.sql.*;`
2. `import java.io.*;`
3. `class RS{`
4. `public static void main(String args[])throws Exception{`
5. `Class.forName("oracle.jdbc.driver.OracleDriver");`
6. `Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system",`
`"oracle");`
- 7.
8. `PreparedStatement ps=con.prepareStatement("insert into emp130 values(?,?,?)");`
- 9.
10. `BufferedReader br=new BufferedReader(new InputStreamReader(System.in));`

```
11.
12. do{
13. System.out.println("enter id:");
14. int id=Integer.parseInt(br.readLine());
15. System.out.println("enter name:");
16. String name=br.readLine();
17. System.out.println("enter salary:");
18. float salary=Float.parseFloat(br.readLine());
19.
20. ps.setInt(1,id);
21. ps.setString(2,name);
22. ps.setFloat(3,salary);
23. int i=ps.executeUpdate();
24. System.out.println(i+" records affected");
25.
26. System.out.println("Do you want to continue: y/n");
27. String s=br.readLine();
28. if(s.startsWith("n")){
29. break;
30. }
31. }while(true);
32.
33. con.close();
34. }}
```

Java ResultSetMetaData Interface

1. The metadata means data about data i.e. we can get further information from the data.
2. If you have to get metadata of a table like total number of column, column name, column type etc. , ResultSetMetaData interface is useful because it provides methods to get metadata from the ResultSet object.

Commonly used methods of ResultSetMetaData interface

Method	Description
public int getColumnCount()throws SQLException	it returns the total number of columns in the ResultSet object.
public String getColumnName(int index)throws SQLException	it returns the column name of the specified column index.
public String getColumnTypeName(int index)throws SQLException	it returns the column type name for the specified index.
public String getTableName(int index)throws SQLException	it returns the table name for the specified column index.

How to get the object of ResultSetMetaData:

The getMetaData() method of ResultSet interface returns the object of ResultSetMetaData. Syntax:

1. **public** ResultSetMetaData getMetaData()**throws** SQLException

Example of ResultSetMetaData interface :

1. **import** java.sql.*;
2. **class** Rsmd{
3. **public static void** main(String args[]){
4. **try**{
5. Class.forName("oracle.jdbc.driver.OracleDriver");
6. Connection con=DriverManager.getConnection(
7. "jdbc:oracle:thin:@localhost:1521:xe","system","oracle");
- 8.
9. PreparedStatement ps=con.prepareStatement("select * from emp");
10. ResultSet rs=ps.executeQuery();
11. ResultSetMetaData rsmd=rs.getMetaData();
- 12.
13. System.out.println("Total columns: "+rsmd.getColumnCount());

```
14. System.out.println("Column Name of 1st column: "+rsmd.getColumnName(1));
15. System.out.println("Column Type Name of 1st column: "+rsmd.getColumnTypeName(1));
16.
17. con.close();
18. }catch(Exception e){ System.out.println(e);}
19. }
20. }
```

```
Output:Total columns: 2
       Column Name of 1st column: ID
       Column Type Name of 1st column: NUMBER
```

Java DatabaseMetaData interface

DatabaseMetaData interface provides methods to get meta data of a database such as database product name, database product version, driver name, name of total number of tables, name of total number of views etc.

Commonly used methods of DatabaseMetaData interface

- **public String getDriverName()throws SQLException:** it returns the name of the JDBC driver.
- **public String getDriverVersion()throws SQLException:** it returns the version number of the JDBC driver.
- **public String getUsername()throws SQLException:** it returns the username of the database.
- **public String getDatabaseProductName()throws SQLException:** it returns the product name of the database.
- **public String getDatabaseProductVersion()throws SQLException:** it returns the product version of the database.
- **public ResultSet getTables(String catalog, String schemaPattern, String tableNamePattern, String[] types)throws SQLException:** it returns the description of the tables of the specified catalog. The table type can be TABLE, VIEW, ALIAS, SYSTEM TABLE, SYNONYM etc.

How to get the object of DatabaseMetaData:

The getMetaData() method of Connection interface returns the object of DatabaseMetaData.

Syntax:

1. **public** DatabaseMetaData getMetaData()**throws** SQLException

Simple Example of DatabaseMetaData interface :

1. **import** java.sql.*;
2. **class** Dbmd{
3. **public static void** main(String args[]){
4. **try**{
5. Class.forName("oracle.jdbc.driver.OracleDriver");
- 6.
7. Connection con=DriverManager.getConnection(
8. "jdbc:oracle:thin:@localhost:1521:xe","system","oracle");
9. DatabaseMetaData dbmd=con.getMetaData();
- 10.
11. System.out.println("Driver Name: "+dbmd.getDriverName());
12. System.out.println("Driver Version: "+dbmd.getDriverVersion());
13. System.out.println("UserName: "+dbmd.getUserName());
14. System.out.println("Database Product Name: "+dbmd.getDatabaseProductName());
15. System.out.println("Database Product Version: "+dbmd.getDatabaseProductVersion());
- 16.
17. con.close();
18. }**catch**(Exception e){ System.out.println(e);}
19. }
20. }

Output:Driver Name: Oracle JDBC Driver

Driver Version: 10.2.0.1.0XE

Database Product Name: Oracle

Database Product Version: Oracle Database 10g Express Edition

Release 10.2.0.1.0 -Production

Example of DatabaseMetaData interface that prints total number of tables :

```
1. import java.sql.*;
2. class Dbmd2{
3.     public static void main(String args[]){
4.         try{
5.             Class.forName("oracle.jdbc.driver.OracleDriver");
6.
7.             Connection con=DriverManager.getConnection(
8.                 "jdbc:oracle:thin:@localhost:1521:xe","system","oracle");
9.
10.            DatabaseMetaData dbmd=con.getMetaData();
11.            String table[]={ "TABLE" };
12.            ResultSet rs=dbmd.getTables(null,null,null,table);
13.
14.            while(rs.next()){
15.                System.out.println(rs.getString(3));
16.            }
17.
18.            con.close();
19.
20.        }catch(Exception e){ System.out.println(e);}
21.
22.    }
23. }
```

Example of DatabaseMetaData interface that prints total number of views :

```
1. import java.sql.*;
2. class Dbmd3{
3.     public static void main(String args[]){
4.         try{
```

```
5. Class.forName("oracle.jdbc.driver.OracleDriver");
6.
7. Connection con=DriverManager.getConnection(
8. "jdbc:oracle:thin:@localhost:1521:xe","system","oracle");
9.
10. DatabaseMetaData dbmd=con.getMetaData();
11. String table[]={"VIEW"};
12. ResultSet rs=dbmd.getTables(null,null,null,table);
13.
14. while(rs.next()){
15. System.out.println(rs.getString(3));
16. }
17.
18. con.close();
19.
20. }catch(Exception e){ System.out.println(e);}
21.
22. }
23. }
```

Java CallableStatement Interface

CallableStatement interface is used to call the **stored procedures and functions**.

We can have business logic on the database by the use of stored procedures and functions that will make the performance better because these are precompiled.

Suppose you need the get the age of the employee based on the date of birth, you may create a function that receives date as the input and returns age of the employee as the output.

What is the difference between stored procedures and functions.

The differences between stored procedures and functions are given below:

Stored Procedure	Function
is used to perform business logic.	is used to perform calculation.
must not have the return type.	must have the return type.
may return 0 or more values.	may return only one values.
We can call functions from the procedure.	Procedure cannot be called from function.
Procedure supports input and output parameters.	Function supports only input parameter.
Exception handling using try/catch block can be used in stored procedures.	Exception handling using try/catch can't be used in user defined functions.

How to get the instance of CallableStatement?

The prepareCall() method of Connection interface returns the instance of CallableStatement. Syntax is given below:

1. **public** CallableStatement prepareCall("{ call procedurename(?,?,...?)}");

The example to get the instance of CallableStatement is given below:

1. CallableStatement stmt=con.prepareCall("{call myprocedure(?,?)}");

It calls the procedure myprocedure that receives 2 arguments.

Transaction Management in JDBC

Transaction represents **a single unit of work**.

The ACID properties describes the transaction management well. ACID stands for Atomicity, Consistency, isolation and durability.

Atomicity means either all successful or none.

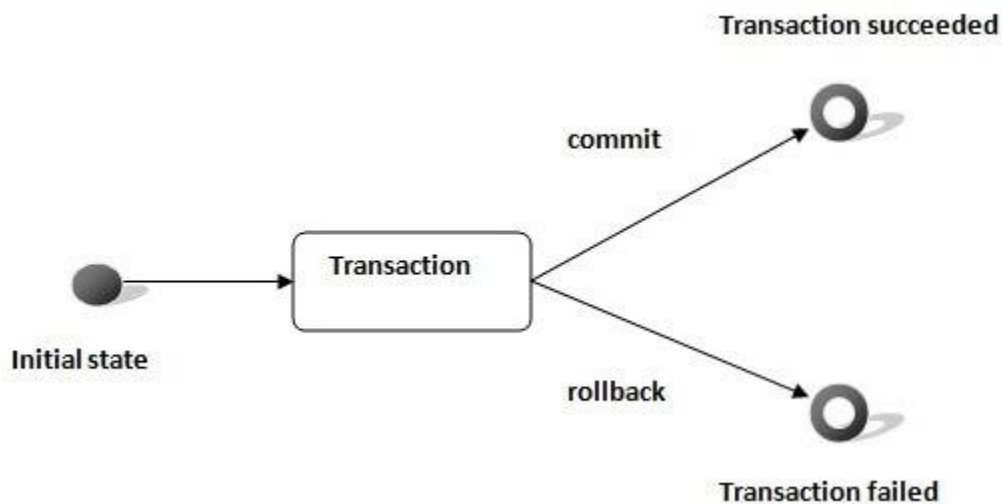
Consistency ensures bringing the database from one consistent state to another consistent state.

Isolation ensures that transaction is isolated from other transaction.

Durability means once a transaction has been committed, it will remain so, even in the event of errors, power loss etc.

Advantage of Transaction Management

fast performance It makes the performance fast because database is hit at the time of commit.



In JDBC, **Connection interface** provides methods to manage transaction.

Method	Description
void setAutoCommit(boolean status)	It is true by default means each transaction is committed by default.
void commit()	commits the transaction.
void rollback()	cancels the transaction.

Simple example of transaction management in jdbc using Statement

Let's see the simple example of transaction management using Statement.

```

1. import java.sql.*;
2. class FetchRecords{
3. public static void main(String args[])throws Exception{
4. Class.forName("oracle.jdbc.driver.OracleDriver");
5. Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system",
   "oracle");
6. con.setAutoCommit(false);
7.
8. Statement stmt=con.createStatement();
9. stmt.executeUpdate("insert into user420 values(190,'abhi',40000)");
10. stmt.executeUpdate("insert into user420 values(191,'umesh',50000)");
11.
12. con.commit();
13. con.close();
14. }}

```

If you see the table emp400, you will see that 2 records has been added.

Example of transaction management in jdbc using PreparedStatement

Let's see the simple example of transaction management using PreparedStatement.

```

1. import java.sql.*;
2. import java.io.*;
3. class TM{
4. public static void main(String args[]){
5. try{
6.
7. Class.forName("oracle.jdbc.driver.OracleDriver");
8. Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system",
   "oracle");
9. con.setAutoCommit(false);
10.
11. PreparedStatement ps=con.prepareStatement("insert into user420 values(?,?,?)");
12.
13. BufferedReader br=new BufferedReader(new InputStreamReader(System.in));

```

```
14. while(true){
15.
16. System.out.println("enter id");
17. String s1=br.readLine();
18. int id=Integer.parseInt(s1);
19.
20. System.out.println("enter name");
21. String name=br.readLine();
22.
23. System.out.println("enter salary");
24. String s3=br.readLine();
25. int salary=Integer.parseInt(s3);
26.
27. ps.setInt(1,id);
28. ps.setString(2,name);
29. ps.setInt(3,salary);
30. ps.executeUpdate();
31.
32. System.out.println("commit/rollback");
33. String answer=br.readLine();
34. if(answer.equals("commit")){
35. con.commit();
36. }
37. if(answer.equals("rollback")){
38. con.rollback();
39. }
40.
41.
42. System.out.println("Want to add more records y/n");
43. String ans=br.readLine();
44. if(ans.equals("n")){
45. break;
46. }
47.
```

```

48. }
49. con.commit();
50. System.out.println("record successfully saved");
51.
52. con.close();//before closing connection commit() is called
53. }catch(Exception e){System.out.println(e);}
54.
55. }}

```

It will ask to add more records until you press n. If you press n, transaction is committed.

Batch Processing in JDBC

Instead of executing a single query, we can execute a batch (group) of queries. It makes the performance fast. It is because when one sends multiple statements of SQL at once to the database, the communication overhead is reduced significantly, as one is not communicating with the database frequently, which in turn results to fast performance.

The `java.sql.Statement` and `java.sql.PreparedStatement` interfaces provide methods for batch processing.

Advantage of Batch Processing

Fast Performance

Methods of Statement interface

The required methods for batch processing are given below:

Method	Description
<code>void addBatch(String query)</code>	The <code>addBatch(String query)</code> method of the <code>CallableStatement</code> , <code>PreparedStatement</code> , and <code>Statement</code> is used to single statements to a batch.
<code>int[] executeBatch()</code>	The <code>executeBatch()</code> method begins the execution of all the grouped together statements. The method returns an integer array, and each of the element of the array represents the updated count for respective

	update statement.
boolean DatabaseMetaData.supportsBatchUpdates() throws SQLException	If the target database facilitates the batch update processing, then the method returns true.
void clearBatch()	The method removes all the statements that one has added using the addBatch() method.

Example of batch processing in JDBC

Let's see the simple example of batch processing in JDBC. It follows following steps:

- Load the driver class
- Create Connection
- Create Statement
- Add query in the batch
- Execute Batch
- Close Connection

FileName: FetchRecords.java

1. **import** java.sql.*;
2. **class** FetchRecords{
3. **public static void** main(String args[])**throws** Exception{
4. Class.forName("oracle.jdbc.driver.OracleDriver");
5. Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system",
"oracle");
6. con.setAutoCommit(**false**);
- 7.
8. Statement stmt=con.createStatement();
9. stmt.addBatch("insert into user420 values(190,'abhi',40000)");
10. stmt.addBatch("insert into user420 values(191,'umesh',50000)");
- 11.
12. stmt.executeBatch();//executing the batch
- 13.

```
14. con.commit();
15. con.close();
16. }}
```

If you see the table user420, two records have been added.

Example of batch processing using PreparedStatement

FileName: BP.java

```
1. import java.sql.*;
2. import java.io.*;
3. class BP{
4.     public static void main(String args[]){
5.     try{
6.
7.         Class.forName("oracle.jdbc.driver.OracleDriver");
8.         Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system",
           "oracle");
9.
10.        PreparedStatement ps=con.prepareStatement("insert into user420 values(?,?,?)");
11.
12.        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
13.        while(true){
14.
15.            System.out.println("enter id");
16.            String s1=br.readLine();
17.            int id=Integer.parseInt(s1);
18.
19.            System.out.println("enter name");
20.            String name=br.readLine();
21.
22.            System.out.println("enter salary");
23.            String s3=br.readLine();
24.            int salary=Integer.parseInt(s3);
```

```
25.  
26. ps.setInt(1,id);  
27. ps.setString(2,name);  
28. ps.setInt(3,salary);  
29.  
30. ps.addBatch();  
31. System.out.println("Want to add more records y/n");  
32. String ans=br.readLine();  
33. if(ans.equals("n")){  
34. break;  
35. }  
36.  
37. }  
38. ps.executeBatch();// for executing the batch  
39.  
40. System.out.println("record successfully saved");  
41.  
42. con.close();  
43. }catch(Exception e){System.out.println(e);}  
44.  
45. }}
```

Output:

```
enter id  
101  
enter name  
Manoj Kumar  
enter salary  
10000  
Want to add more records y/n  
y  
enter id
```

```
101
enter name
Harish Singh
enter salary
15000
Want to add more records y/n
y
enter id
103
enter name
Rohit Anuragi
enter salary
30000
Want to add more records y/n
y
enter id
104
enter name
Amrit Gautam
enter salary
40000
Want to add more records y/n
n
record successfully saved
```

It will add the queries into the batch until user press n. Finally, it executes the batch. Thus, all the added queries will be fired.

JDBC RowSet

An instance of **RowSet** is the Java bean component because it has properties and Java bean notification mechanism. It is the wrapper of **ResultSet**. A JDBC RowSet facilitates a mechanism to keep the data in tabular form. It happens to make the data more flexible as well as easier as

compared to a ResultSet. The connection between the data source and the RowSet object is maintained throughout its life cycle. The RowSet supports development models that are component-based such as JavaBeans, with the standard set of properties and the mechanism of event notification.

It was in the JDBC 2.0, the support for the RowSet was introduced using the optional packages. But the implementations were standardized for RowSet in the JDBC RowSet Implementations Specification (JSR-114) by the Sun Microsystems that is being present in the JDK (Java Development Kit) 5.0.

The implementation classes of the RowSet interface are as follows:

- JdbcRowSet
- CachedRowSet
- WebRowSet
- JoinRowSet
- FilteredRowSet

1. JdbcRowSet rowSet = RowSetProvider.newFactory().createJdbcRowSet();
2. rowSet.setUrl("jdbc:oracle:thin:@localhost:1521:xe");
3. rowSet.setUsername("system");
4. rowSet.setPassword("oracle");
- 5.
6. rowSet.setCommand("select * from emp400");
7. rowSet.execute();

Advantage of RowSet

The advantages of using RowSet are given below:

1. It is easy and flexible to use.
2. It is Scrollable and Updatable by default.

Example of JdbcRowSet

Let's see the simple example of JdbcRowSet without event handling code.

FileName: RowSetExample.java

1. **import** java.sql.Connection;

```

2. import java.sql.DriverManager;
3. import java.sql.ResultSet;
4. import java.sql.Statement;
5. import javax.sql.RowSetEvent;
6. import javax.sql.RowSetListener;
7. import javax.sql.rowset.JdbcRowSet;
8. import javax.sql.rowset.RowSetProvider;
9.
10. public class RowSetExample {
11.     public static void main(String[] args) throws Exception {
12.         Class.forName("oracle.jdbc.driver.OracleDriver");
13.
14.         //Creating and Executing RowSet
15.         JdbcRowSet rowSet = RowSetProvider.newFactory().createJdbcRowSet();
16.         rowSet.setUrl("jdbc:oracle:thin:@localhost:1521:xe");
17.         rowSet.setUsername("system");
18.         rowSet.setPassword("oracle");
19.
20.         rowSet.setCommand("select * from emp400");
21.         rowSet.execute();
22.
23.         while (rowSet.next()) {
24.             // Generating cursor Moved event
25.             System.out.println("Id: " + rowSet.getString(1));
26.             System.out.println("Name: " + rowSet.getString(2));
27.             System.out.println("Salary: " + rowSet.getString(3));
28.         }
29.
30.     }
31. }

```

The output is given below:

```
Id: 55
Name: Om Bhim
Salary: 70000
Id: 190
Name: abhi
Salary: 40000
Id: 191
Name: umesh
Salary: 50000
```

Example of JDBC RowSet with Event Handling

To perform event handling with `JdbcRowSet`, you need to add the instance of **RowSetListener** in the `addRowSetListener` method of `JdbcRowSet`.

The `RowSetListener` interface provides 3 method that must be implemented. They are as follows:

1. `public void cursorMoved(RowSetEvent event);`
2. `public void rowChanged(RowSetEvent event);`
3. `public void rowSetChanged(RowSetEvent event);`

Let's write the code to retrieve the data and perform some additional tasks while the cursor is moved, the cursor is changed, or the rowset is changed. The event handling operation can't be performed using `ResultSet`, so it is preferred now.

FileName: RowSetExample.java

1. **import** java.sql.Connection;
2. **import** java.sql.DriverManager;
3. **import** java.sql.ResultSet;
4. **import** java.sql.Statement;
5. **import** javax.sql.RowSetEvent;
6. **import** javax.sql.RowSetListener;
7. **import** javax.sql.rowset.JdbcRowSet;
8. **import** javax.sql.rowset.RowSetProvider;
- 9.
10. **public class** RowSetExample {
11. **public static void** main(String[] args) **throws** Exception {

```
12.         Class.forName("oracle.jdbc.driver.OracleDriver");
13.
14.     //Creating and Executing RowSet
15.     JdbcRowSet rowSet = RowSetProvider.newFactory().createJdbcRowSet();
16.     rowSet.setUrl("jdbc:oracle:thin:@localhost:1521:xe");
17.     rowSet.setUsername("system");
18.     rowSet.setPassword("oracle");
19.
20.     rowSet.setCommand("select * from emp400");
21.     rowSet.execute();
22.
23.     //Adding Listener and moving RowSet
24.     rowSet.addRowSetListener(new MyListener());
25.
26.         while (rowSet.next()) {
27.             // Generating cursor Moved event
28.             System.out.println("Id: " + rowSet.getString(1));
29.             System.out.println("Name: " + rowSet.getString(2));
30.             System.out.println("Salary: " + rowSet.getString(3));
31.         }
32.
33.     }
34. }
35.
36. class MyListener implements RowSetListener {
37.     public void cursorMoved(RowSetEvent event) {
38.         System.out.println("Cursor Moved...");
39.     }
40.     public void rowChanged(RowSetEvent event) {
41.         System.out.println("Cursor Changed...");
42.     }
```

```
43. public void rowSetChanged(RowSetEvent event) {  
44.     System.out.println("RowSet changed...");  
45. }  
46. }
```

The output is as follows:

```
Cursor Moved...  
Id: 55  
Name: Om Bhim  
Salary: 70000  
Cursor Moved...  
Id: 190  
Name: abhi  
Salary: 40000  
Cursor Moved...  
Id: 191  
Name: umesh  
Salary: 50000  
Cursor Moved...
```

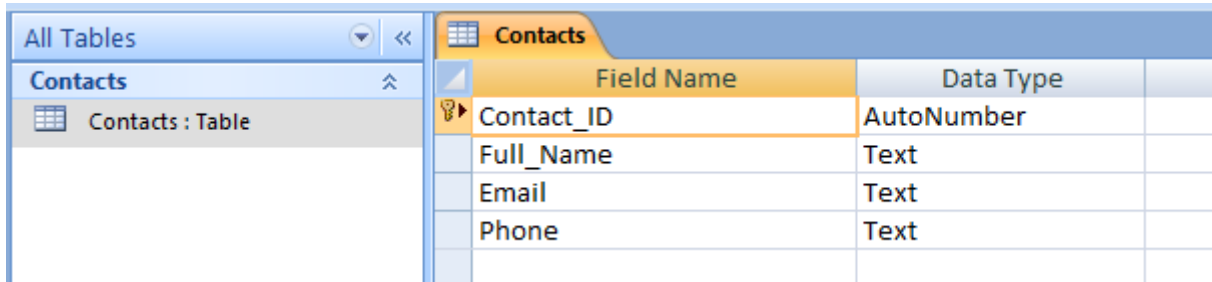
Java JDBC Example Connect to Microsoft Access Database

There are several third-party [JDBC drivers](#) out there for Microsoft Access database, and we recommend **UCanAccess** - a pure Java JDBC Driver for Access that allows Java developers and JDBC client programs to read/write Microsoft Access databases. UCanAccess supports various Access formats: 2000, 2002/2003, 2007, 2010/2013/2016 (Access 97 is supported for read-only). UCanAccess is open-source and implemented entirely in Java so it can be used across platforms (Windows, Mac, Linux...). It also provides Maven dependencies so you can integrate it in your existing projects quickly.

In case you don't use Maven, you have to [download UCanAccess](#) distribution and add the following JAR files to the classpath:

- ucanaccess-4.0.4.jar
- hsqldb-2.3.1.jar
- jackcess-2.1.11.jar
- commons-lang-2.6.jar
- commons-logging-1.1.3.jar

The version numbers here may differ than the latest versions you downloaded. Now, let's see how to write a simple Java program to read/write a Microsoft Access database.



Field Name	Data Type
Contact_ID	AutoNumber
Full_Name	Text
Email	Text
Phone	Text

The database file is located at **e:\Java\JavaSE\MsAccess\Contacts.accdb**. - This path will be used in database URL. We will write a Java program that uses the UCanAccess JDBC driver to connect to this database, insert a row and select all rows from the table **Contacts**.

You can use JDBC API as normal (see [Connect to a database with JDBC](#)). The differences lie in the database URL and Access-specific SQL syntax you can use. For example, you need to construct the database URL to include path of the Access database file like this:

```
1 String databaseURL = "jdbc:ucanaccess://e://Java//JavaSE//MsAccess//Contacts.accdb";
```

And here is code of the example program:

```

import java.sql.*;

public class JavaApplication2 {

    /**
     * @param args the command line arguments
     */

    public static void main(String[] args) {
        // TODO code application logic here

        String databaseURL = "jdbc:ucanaccess://C:\\Users\\admin\\Documents\\Database1.accdb";

        try (Connection connection = DriverManager.getConnection(databaseURL)) {

            String sql = "INSERT INTO table1 VALUES (?, ?, ?)";

            PreparedStatement preparedStatement = connection.prepareStatement(sql);
            preparedStatement.setInt(1, 101);
            preparedStatement.setString(2, "Ankit");
            preparedStatement.setInt(3, 25);

            int row = preparedStatement.executeUpdate();

            if (row > 0) {
                System.out.println("A row has been inserted successfully.");
            }

            sql = "SELECT * FROM table1";

            Statement statement = connection.createStatement();
            ResultSet result = statement.executeQuery(sql);

```

```
while (result.next()) {  
    int id = result.getInt(1);  
    String fullname = result.getString(2);  
    int age = result.getInt(3);  
  
    System.out.println(id + ", " + fullname + ", " + age );  
}  
  
} catch (SQLException ex)  
{  
    System.out.println(ex);  
}  
}  
}
```